LEARNING SESSION S5

Python: Basic to Advanced (for non-tech learners)

Learn Python step by step with everyday business examples: from your first line of code to pandas, charts, automation and ML

AnalytixLabs · Advanced Certification in Data Science & AI (IIT), June 2026 batch

Interactive version (every tab clickable):
https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw

Contents

Start here

Level 1 · Basics

Level 2 · Data structures

Level 3 · Functions, files, errors

Level 4 · pandas

Level 5 · Charts

Level 6 · Stats & ML

Level 7 · Automation

Level 8 · Advanced

Interview Q&A

Resources

LEARNING SESSION S5

🐍 Python: Basic to Advanced (for non-tech learners)

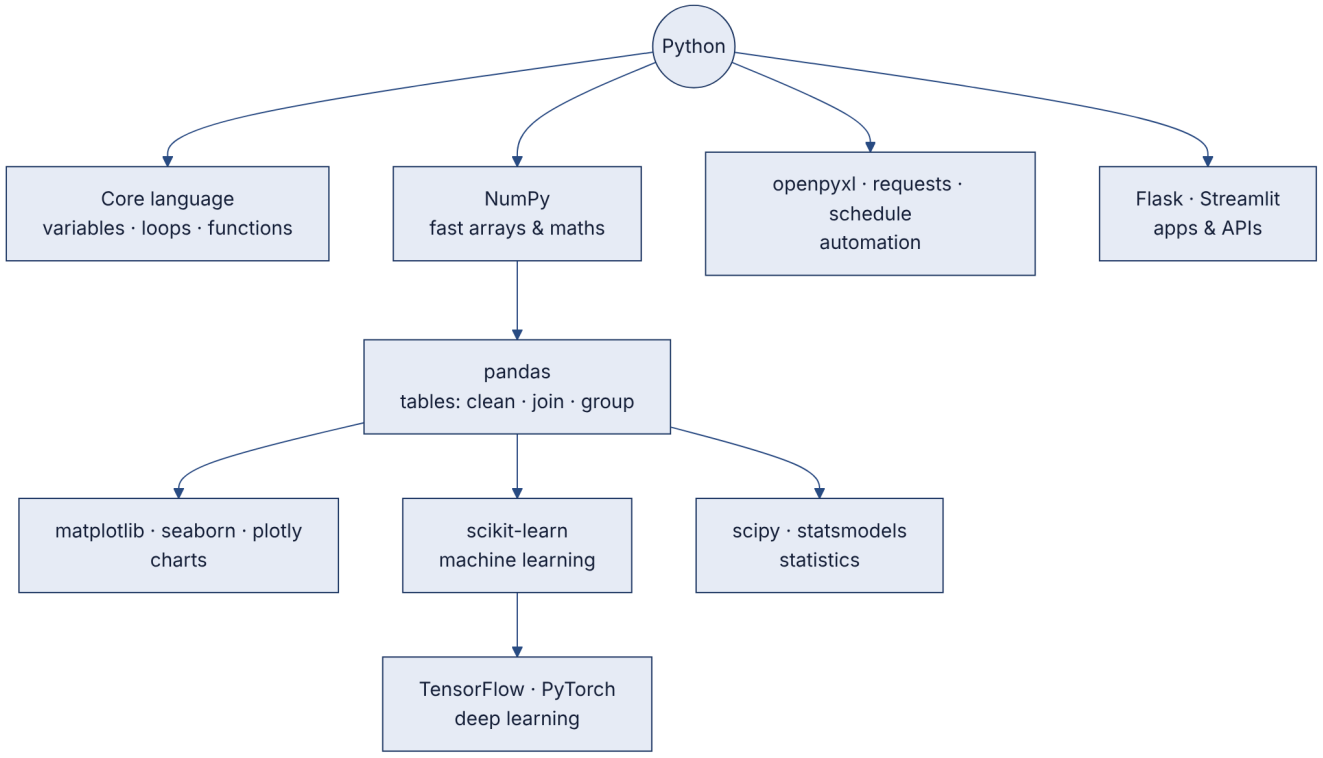
Learn Python step by step with everyday business examples: from your first line of code to pandas, charts, automation and ML

Learning session · no programming background needed · runs in Google Colab or Jupyter

Open this module online: **https://claude.ai/artifact/GtUBqgmq7C6gvyYa7LjhEw#s5**

Start here

If you can write an Excel formula, you can learn Python. Python is a set of instructions written in near-English that the computer follows exactly. Analysts use it because it repeats work perfectly, handles millions of rows, and connects Excel, databases, the web and AI.



Core Python first, then the libraries that make it a data tool.

Set up in 5 minutes

- **Easiest**: open Google Colab with a Google account → New notebook. Nothing to install.
- **On your laptop**: install Anaconda (includes Python, Jupyter, pandas) → launch Jupyter Notebook; or install Python + VS Code.
- Run a cell with `Shift` + `Enter`.

Excel → Python translation

Excel	Python (pandas)
Open a file	`df = pd.read_excel("sales.xlsx")`
Filter	`df[df["City"] == "Pune"]`
Sort	`df.sort_values("Amount", ascending=False)`
New column =C2*D2	`df["Total"] = df["Qty"] * df["Price"]`
SUMIFS / pivot	`df.groupby("City")["Amount"].sum()` / `df.pivot_table(...)`
VLOOKUP	`df.merge(products, on="ProductID", how="left")`
Remove duplicates	`df.drop_duplicates()`
Chart	`df.plot.bar(x="City", y="Amount")`

30-day plan

Days 1–7	Levels 1–2 (basics, data structures), 30 minutes daily
Days 8–12	Level 3 (functions, files, errors)
Days 13–22	Levels 4–5 (pandas + charts) on your own Excel files
Days 23–27	Level 7 automation: automate one weekly report
Days 28–30	Mini project: clean → analyse → chart → one-page summary

Level 1 · Basics

```
print("Hello, Data!")              # show something
name = "Asha"                      # variable = a labelled box holding a value
orders = 12                        # int (whole number)
avg_value = 1310.50                # float (decimal)
is_prime = True                    # bool (True/False)
print(f"{name} placed {orders} orders worth ₹{orders * avg_value:,.0f}")   # f-string
# Asha placed 12 orders worth ₹15,726

type(avg_value)                    # float
int("42") + 1                      # 43  convert text to number
round(1310.567, 1)                 # 1310.6

city = input("Enter city: ")       # ask the user (always returns text)

# Decisions
amount = 7200
if amount > 5000:
    print("Free delivery")
elif amount > 2000:
    print("₹49 delivery")
```

```
else:
    print("₹99 delivery")

# Repetition
for month in ["Jan", "Feb", "Mar"]:
    print("Report for", month)
for i in range(1, 6):              # 1,2,3,4,5
    print(i * 100)
```

> **Tip** Indentation (4 spaces) is part of Python's grammar: it shows which lines belong inside an if or for. A missing colon or wrong indent is the most common beginner error.

Level 2 · Data structures

```
cities = ["Pune", "Delhi", "Mumbai"]          # list: ordered, changeable
cities.append("Chennai"); cities[0]; cities[-1]; len(cities)
sales = {"Pune": 120000, "Delhi": 95000}      # dict: key → value (like a lookup table)
sales["Mumbai"] = 87000
for city, value in sales.items():
    print(city, value)
point = (28.46, 77.03)                        # tuple: fixed (latitude, longitude)
unique_ids = set([101, 102, 101])             # set: unique values → {101, 102}

# List comprehension: build a list in one line
amounts = [450, 7200, 3100, 980]
big = [a for a in amounts if a > 1000]         # [7200, 3100]
with_gst = [round(a * 1.18) for a in amounts]  # add 18% GST

# Strings
email = "  Asha.Verma@Example.com "
email.strip().lower()                          # 'asha.verma@example.com'
email.split("@")[1]                            # domain part
"Pune" in "Pune, Maharashtra"                  # True
```

Level 3 · Functions, files, errors

```
def delivery_fee(amount, is_prime=False):
    """Return the delivery fee for an order."""
    if is_prime or amount > 5000:
        return 0
    return 49 if amount > 2000 else 99

delivery_fee(1500)          # 99
delivery_fee(1500, True)    # 0

gst = lambda x: x * 0.18    # small one-line function
list(map(gst, [100, 200]))  # [18.0, 36.0]

# Files
with open("notes.txt", "w") as f:
```

```
    f.write("Report generated\n")
import csv, os, datetime
os.listdir(".")            # files in the folder

# Errors: handle them instead of crashing
try:
    rate = int("18%")
except ValueError:
    rate = 18
    print("Could not read the rate; using 18")
```

Modules: `import math`, `from datetime import date`, `import pandas as pd`. Install new ones with `pip install package-name` (in Colab: `!pip install ...`).

Level 4 · pandas

A complete mini-analysis

```
import pandas as pd
df = pd.read_csv("orders.csv", parse_dates=["order_date"])   # or read_excel / read_sql

df.shape                      # (rows, columns)
df.head()                     # first 5 rows
df.info()                     # columns, types, missing values
df.describe()                 # quick statistics

# Clean
df = df.drop_duplicates()
df["city"] = df["city"].str.strip().str.title()
df["amount"] = pd.to_numeric(df["amount"], errors="coerce")
df["amount"] = df["amount"].fillna(df["amount"].median())

# New columns
df["month"] = df["order_date"].dt.to_period("M")
df["band"] = pd.cut(df["amount"], [0, 2000, 10000, float("inf")], labels=["Low", "Medium",
"High"])

# Questions
df[df["city"] == "Pune"]["amount"].sum()                          # Pune revenue
df.groupby("city")["amount"].agg(["count", "sum", "mean"]).sort_values("sum", ascending=False)
df.pivot_table(index="city", columns="month", values="amount", aggfunc="sum", fill_value=0)
top5 = df.groupby("customer_id")["amount"].sum().nlargest(5)

# Combine with another table (VLOOKUP)
products = pd.read_excel("products.xlsx")
df = df.merge(products[["product_id", "category"]], on="product_id", how="left")

# Save
df.to_excel("clean_orders.xlsx", index=False)
```

Full pandas reference: **Python module → pandas**. 90 practice exercises solved: **Data Manipulation exercise**.

Level 5 · Charts

```python
import matplotlib.pyplot as plt
import seaborn as sns

monthly = df.groupby("month")["amount"].sum()
monthly.plot(kind="line", marker="o", title="Monthly revenue")
plt.ylabel("₹"); plt.show()

df.groupby("city")["amount"].sum().sort_values().plot(kind="barh", title="Revenue by city")
sns.histplot(df["amount"], bins=30)                        # distribution
sns.boxplot(data=df, x="city", y="amount")                 # spread and outliers
sns.heatmap(df.corr(numeric_only=True), annot=True)       # correlations

import plotly.express as px                                # interactive charts
px.bar(df, x="city", y="amount", color="band").show()
```

Level 6 · Stats & ML

```python
from scipy import stats
a = df[df.city == "Pune"]["amount"]; b = df[df.city == "Delhi"]["amount"]
stats.ttest_ind(a, b, equal_var=False)          # do Pune and Delhi spend differently?

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
X = df[["amount", "orders_last_90d", "days_since_last"]]
y = df["churned"]                                # 1 = left, 0 = stayed
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=1)
model = LogisticRegression(max_iter=1000).fit(X_tr, y_tr)
print(classification_report(y_te, model.predict(X_te)))
```

Explained without jargon in Machine Learning for non-tech.

Level 7 · Automation

```python
# Combine every monthly Excel file in a folder into one
import glob
files = glob.glob("reports/sales_*.xlsx")
all_sales = pd.concat([pd.read_excel(f).assign(source=f) for f in files], ignore_index=True)

# Write a formatted Excel report with several sheets
with pd.ExcelWriter("weekly_report.xlsx") as xw:
    all_sales.groupby("city")["amount"].sum().to_excel(xw, sheet_name="By city")
    all_sales.to_excel(xw, sheet_name="Data", index=False)

# Get data from a web API
import requests
rates = requests.get("https://api.frankfurter.app/latest?from=USD&to=INR").json()

# Read from a database
```

```
import sqlalchemy as sa
engine = sa.create_engine("mssql+pyodbc://@localhost/RetailDB?
driver=ODBC+Driver+18+for+SQL+Server&trusted_connection=yes&TrustServerCertificate=yes")
orders = pd.read_sql("SELECT * FROM Orders WHERE order_date >= '2026-10-01'", engine)
```

Schedule the script with Windows Task Scheduler, cron, or a cloud scheduler; see **Automation on Data**.

Level 8 · Advanced

OOP (classes)	Bundle data + behaviour: `class Report: def __init__(self, df): ...`
Comprehensions & generators	`{k: v for k, v in ...}`; `yield` to stream large files
Decorators	Wrap functions (timing, caching: `@functools.lru_cache`)
Vectorisation	Use pandas/NumPy column operations instead of loops (10–100× faster)
Virtual environments	`python -m venv .venv` or conda; pin versions in requirements.txt
Git & GitHub	Version control and portfolio (see ML Ops module)
Testing	`pytest` for functions that compute money
Type hints & linting	`def fee(amount: float) -> int:`; ruff/black for clean code
Big data	Polars, DuckDB, PySpark when pandas runs out of memory
Apps	Streamlit dashboards and Flask/FastAPI APIs

Coding with AI assistants

- Ask for small pieces with context: "pandas DataFrame df has columns city, amount, order_date. Write code for monthly revenue by city as a pivot table."
- Ask it to explain errors: paste the code and the full error message.
- Use Colab's AI, GitHub Copilot, Cursor or Claude Code inside the editor; review every line before trusting it.

Interview Q&A

Q List vs tuple vs dict vs set?

List: ordered, changeable. Tuple: ordered, fixed. Dict: key-value lookups. Set: unique values, fast membership tests.

Q How do you handle missing values in pandas?

Find them with `isna().sum()`; drop with `dropna` or fill with `fillna` (median for skewed numbers, mode for categories), or flag them.

Q merge vs concat?

merge joins on key columns (like VLOOKUP/SQL JOIN); concat stacks DataFrames on top of or beside each other.

Q What does groupby do?

Split rows into groups, apply an aggregation to each, combine the results: like SUMIFS or a pivot table.

Q Why avoid loops over DataFrame rows?

They are slow; vectorised column operations run in optimised C code.

Q What is a virtual environment?

An isolated set of packages for one project, so versions don't clash between projects.

Resources

- ▶ YouTube: Simplilearn Python full course · ▶ YouTube: Simplilearn Python for data science · ▶ YouTube: Simplilearn pandas tutorial
- ▶ YouTube: Corey Schafer Python beginner tutorial · ▶ YouTube: Keith Galli pandas tutorial · ▶ YouTube: freeCodeCamp Python for everybody
- ▶ YouTube: Automate the Boring Stuff with Python
- 📘 Google Colab · 📘 Python tutorial · 📘 pandas: getting started
- 📘 Kaggle Learn: Python and pandas (free) · 📘 Automate the Boring Stuff (free online book)